

HW11

December 5, 2025

1 Activity 1. Data Loading and Processing

1.1 1. Load the Dataset

```
[1]: import pandas as pd
import numpy as np

np.random.seed(42)  # set random seed

users = [f"U{str(i).zfill(3)}" for i in range(1, 21)]
items = [f"S{str(i).zfill(3)}" for i in range(1, 31)]

# generate data
num_rows = 100
df = pd.DataFrame({
    "user_id": np.random.choice(users, num_rows),
    "item_id": np.random.choice(items, num_rows),
    "listen_count": np.random.randint(1, 6, num_rows)
})

df.head()
```

```
[1]:  user_id item_id  listen_count
0     U007     S003             4
1     U020     S012             4
2     U015     S008             2
3     U011     S022             3
4     U008     S027             1
```

1.2 2. Explore the dataset

- a) Check the data shape
- b) Display the first 3 rows

```
[2]: # a) Check the data shape
print("Data shape (rows, columns):", df.shape)
```

```
# b) Display the first 3 rows
df.head(3)
```

Data shape (rows, columns): (100, 3)

```
[2]:  user_id item_id listen_count
      0    U007    S003           4
      1    U020    S012           4
      2    U015    S008           2
```

2 Activity 2. Popularity-Based Recommendation

1. Compute the total listen count for each song (item)
2. Display the top 10 most frequently played songs

```
[3]: # 1. Compute the total listen count for each song (item)
item_popularity = (
    df.groupby('item_id')['listen_count']
      .sum()
      .sort_values(ascending=False)
      .reset_index(name='total_listen_count')
)

# 2. Display the top 10 most frequently played songs
top_10_songs = item_popularity.head(10)
top_10_songs
```

```
[3]:  item_id total_listen_count
      0    S030                25
      1    S007                20
      2    S013                19
      3    S005                18
      4    S027                18
      5    S012                13
      6    S003                13
      7    S025                13
      8    S028                11
      9    S015                11
```

3 Activity 3. Item-Based Collaborative Filtering

1. Construct a song co-occurrence matrix
2. Compute an item similarity matrix using cosine similarity
3. Display the top 3 songs similar to song **S003**

```
[7]: # 1. Construct a song co-occurrence matrix
# First build a user-item interaction matrix (binary: listened or not)
user_item_matrix = (
    df.groupby(['user_id', 'item_id'])['listen_count']
      .sum()
      .unstack(fill_value=0)
)

# Convert to binary interactions: 1 if user has listened to the song, else 0
user_item_binary = (user_item_matrix > 0).astype(int)

# Co-occurrence matrix: items x items, counting how many users listened to both
# ↪ songs
co_occurrence_matrix = user_item_binary.T.dot(user_item_binary)
co_occurrence_matrix.head()
```

```
[7]: item_id  S001  S002  S003  S004  S005  S006  S007  S008  S009  S010  ... \
item_id
S001         5     0     1     0     1     0     1     0     1     0  ...
S002         0     1     0     0     0     0     0     0     1     0  ...
S003         1     0     4     1     1     0     1     0     1     0  ...
S004         0     0     1     2     0     0     0     1     0     1  ...
S005         1     0     1     0     4     1     2     1     1     0  ...

item_id  S021  S022  S023  S024  S025  S026  S027  S028  S029  S030
item_id
S001         1     1     1     0     0     0     1     1     0     0
S002         0     0     0     0     0     0     0     0     0     1
S003         0     3     0     1     0     0     2     0     1     2
S004         0     0     1     1     1     1     1     0     0     1
S005         0     1     0     0     0     0     3     1     0     1

[5 rows x 28 columns]
```

```
[8]: # 2. Compute an item similarity matrix using cosine similarity
import numpy as np

# diagonal elements = sqrt(C_ii)
diag = np.sqrt(np.diag(co_occurrence_matrix))

# To avoid division by zero, replace zeros with a very small number
diag[diag == 0] = 1e-10

denominator = np.outer(diag, diag)
similarity_matrix = co_occurrence_matrix / denominator

# Put back into a DataFrame with item_id labels
```

```

item_similarity_df = pd.DataFrame(
    similarity_matrix,
    index=co_occurrence_matrix.index,
    columns=co_occurrence_matrix.columns
)

item_similarity_df.head()

```

```

[8]: item_id      S001  S002      S003      S004      S005      S006      S007  \
item_id
S001      1.000000   0.0  0.223607  0.000000  0.223607  0.000000  0.200000
S002      0.000000   1.0  0.000000  0.000000  0.000000  0.000000  0.000000
S003      0.223607   0.0  1.000000  0.353553  0.250000  0.000000  0.223607
S004      0.000000   0.0  0.353553  1.000000  0.000000  0.000000  0.000000
S005      0.223607   0.0  0.250000  0.000000  1.000000  0.353553  0.447214

item_id      S008      S009      S010  ...      S021      S022      S023  \
item_id
S001      0.000000  0.223607  0.000000  ...  0.447214  0.258199  0.258199
S002      0.000000  0.500000  0.000000  ...  0.000000  0.000000  0.000000
S003      0.000000  0.250000  0.000000  ...  0.000000  0.866025  0.000000
S004      0.500000  0.000000  0.408248  ...  0.000000  0.000000  0.408248
S005      0.353553  0.250000  0.000000  ...  0.000000  0.288675  0.000000

item_id      S024      S025      S026      S027      S028      S029      S030
item_id
S001      0.000000  0.000000  0.000000  0.200000  0.200000  0.000000  0.000000
S002      0.000000  0.000000  0.000000  0.000000  0.000000  0.000000  0.353553
S003      0.353553  0.000000  0.000000  0.447214  0.000000  0.353553  0.353553
S004      0.500000  0.408248  0.353553  0.316228  0.000000  0.000000  0.250000
S005      0.000000  0.000000  0.000000  0.670820  0.223607  0.000000  0.176777

[5 rows x 28 columns]

```

```

[9]: # 3. Display the top 3 songs similar to song S003
target_item = 'S003'

if target_item in item_similarity_df.columns:
    # Get similarity scores for S003, drop itself, sort descending
    similar_scores = (
        item_similarity_df[target_item]
        .drop(labels=[target_item])
        .sort_values(ascending=False)
    )

    top_3_similar = similar_scores.head(3).reset_index()
    top_3_similar.columns = ['item_id', 'similarity_to_S003']

```

```
print("Top 3 songs similar to S003:")
display(top_3_similar)
else:
    print(f"{target_item} not found in the item similarity matrix.")
```

Top 3 songs similar to S003:

	item_id	similarity_to_S003
0	S022	0.866025
1	S015	0.447214
2	S027	0.447214

[]: